# 05. dstack Gateway 네트워킹 시스템 상세 분석

## 목차


1. Gateway 아키텍처 개요
2. 프록시 시스템
3. TLS 처리
4. WireGuard VPN
5. 도메인 라우팅
6. 연결 관리
7. 인증서 관리
8. 클러스터링


---

## 1. Gateway 아키텍처 개요

### 1.1 시스템 구성

```
┌─────────────────────────────────────────────────────────────────────┐
│ Internet │
└──────────────────────┬──────────────────────────────────────────────┘
│ HTTPS/TLS
┌──────────────────────▼──────────────────────────────────────────────┐
```

```
| dstack Gateway |
├──────────────────────────────────────────────────────────────┤
| ┌──────────────┐ ┌──────────────┐ ┌────────────────────┐ |
| | TLS | | Proxy | | WireGuard VPN | |
| | Termination | | Router | | Network | |
| └──────────────┘ └──────────────┘ └────────────────────┘ |
├──────────────────────────────────────────────────────────────┤
| ┌──────────────┐ ┌──────────────┐ ┌────────────────────┐ |
| | Certificate | | Node | | Instance | |
| | Management | | Discovery | | Registry | |
| └──────────────┘ └──────────────┘ └────────────────────┘ |
└──────────────────────┬───────────────────────────────────────┘
| WireGuard Tunnel
┌──────────────────────▼───────────────────────────────────────┐
| CVM Network |
| ┌──────────────┐ ┌──────────────┐ ┌────────────────────┐ |
| | CVM #1 | | CVM #2 | | CVM #N | |
| | (App A) | | (App B) | | (App N) | |
| └──────────────┘ └──────────────┘ └────────────────────┘ |
└──────────────────────────────────────────────────────────────┘
```

## 1.2 핵심 구조체

```rust
// gateway/src/main_service.rs

#[derive(Clone)]

pub  struct Proxy {

_inner: Arc<ProxyInner>,

}



pub  struct ProxyInner {

pub(crate) config: Arc<Config>,

pub(crate) certbot: Option<Arc<CertBot>>,

my_app_id: Option<Vec<u8>>,

state: Mutex<ProxyState>,

notify_state_updated: Notify,

auth_client: AuthClient,

pub(crate) acceptor: RwLock<TlsAcceptor>,

}



#[derive(Debug, Serialize, Deserialize, Default)]

pub(crate) struct ProxyStateMut {

pub(crate) nodes: BTreeMap<String, GatewayNodeInfo>, // 다른 Gateway 노드들

pub(crate) apps: BTreeMap<String, BTreeSet<String>>, // 앱 ID → 인스턴스 ID 매핑

pub(crate) instances: BTreeMap<String, InstanceInfo>, // 인스턴스 정보

pub(crate) allocated_addresses: BTreeSet<Ipv4Addr>, // 할당된 IP 주소들
```

```
pub(crate) top_n: BTreeMap<String, (AddressGroup, Instant)>, // 캐시된 주소 그룹
}
```

---

# 2. 프록시 시스템

---

## 2.1 프록시 메인 루프

```
// gateway/src/proxy.rs
pub  async  fn  proxy_main(config: &ProxyConfig, proxy: Proxy) -> Result<()> {
// 1. 워커 스레드 풀 생성
let  workers_rt = tokio::runtime::Builder::new_multi_thread()
.thread_name("proxy-worker")
.enable_all()
.worker_threads(config.workers)
.build()
.expect("Failed to build Tokio runtime");

// 2. 도메인 설정
let  dotted_base_domain = {
let  base_domain = config.base_domain.as_str();
let  base_domain = base_domain.strip_prefix(".").unwrap_or(base_domain);
Arc::new(format!(".{base_domain}"))
};
```

```
// 3. TCP 리스너 바인딩
let listener = TcpListener::bind((config.listen_addr, config.listen_port))
.await
.with_context(|| {
format!(
"failed to bind {}:{}",
config.listen_addr, config.listen_port
)
})?;
info!(
"tcp bridge listening on {}:{}",
config.listen_addr, config.listen_port
);

// 4. 연결 수락 루프
loop {
match listener.accept().await {
Ok((inbound, from)) => {
let span = info_span!("conn", id = next_connection_id());
let _enter = span.enter();
let conn_entered = EnteredCounter::new(&NUM_CONNECTIONS);

info!(%from, "new connection");
```

```
let proxy = proxy.clone();
let dotted_base_domain = dotted_base_domain.clone();
// 5. 워커 스레드에서 연결 처리
workers_rt.spawn(
async move {
let _conn_entered = conn_entered;
let timeouts = &proxy.config.proxy.timeouts;
let result = timeout(
timeouts.total,
handle_connection(inbound, proxy, &dotted_base_domain),
)
.await;
match result {
Ok(Ok(_)) => {
info!("connection closed");
}
Ok(Err(e)) => {
error!("connection error: {e:?}");
}
Err(_) => {
error!("connection timeout");
}
}
```

```
}
.instrument(span),
);
}
Err(e) => {
error!("failed to accept connection: {e:?}");
}
}
}
}
```

## 2.2 연결 처리 과정

```
// gateway/src/proxy.rs
async fn handle_connection(
mut inbound: TcpStream,
state: Proxy,
dotted_base_domain: &str,
) -> Result<()> {
let timeouts = &state.config.proxy.timeouts;
// 1. SNI (Server Name Indication) 추출
let (sni, buffer) = timeout(timeouts.handshake, take_sni(&mut inbound))
.await
```

```
.context("timeout on handshake")?

.context("failed to take sni")?;



debug!("sni: {sni:?}");

// 2. 도메인 기반 라우팅 결정

if  let Some(dst) = parse_destination(&sni, dotted_base_domain) {

debug!("dst: {dst:?}");

if  dst.is_tls {

// TLS 패스스루 모드

tls_passthough::proxy_to_app(state, inbound, buffer, &dst.app_id, dst.port).awai

} else {

// TLS 종료 모드

state.proxy(inbound, buffer, &dst.app_id, dst.port).await

}

} else {

// 일반 SNI 기반 프록시

tls_passthough::proxy_with_sni(state, inbound, buffer, &sni).await

}

}
```

## 2.3 도메인 파싱

```
// gateway/src/proxy.rs (추정)
```

```rust
#[derive(Debug)]
struct Destination {
app_id: String,
port: u16,
is_tls: bool,
}


fn parse_destination(sni: &str, base_domain: &str) -> Option<Destination> {
// 도메인 형식: <app_id>[-<port>][s].<base_domain>
// 예시:
// - abc123.app.example.com → app_id=abc123, port=80, is_tls=false
// - abc123-8080.app.example.com → app_id=abc123, port=8080, is_tls=false
// - abc123-8080s.app.example.com → app_id=abc123, port=8080, is_tls=true
if !sni.ends_with(base_domain) {
return None;
}
let subdomain = sni.strip_suffix(base_domain)?.strip_suffix(".")?;
// TLS 패스스루 확인 (끝에 's'가 있는지)
let (subdomain, is_tls) = if subdomain.ends_with('s') {
(subdomain.strip_suffix('s')?, true)
} else {
(subdomain, false)
};
```

```
// 포트 분리
let (app_id, port) = if  let Some(dash_pos) = subdomain.rfind('-') {
let  app_id = &subdomain[..dash_pos];
let  port_str = &subdomain[dash_pos + 1..];
let  port = port_str.parse().ok()?;
(app_id.to_string(), port)
} else {
(subdomain.to_string(), if  is_tls { 443 } else { 80 })
};
Some(Destination {
app_id,
port,
is_tls,
})
}
```

---

# 3. TLS 처리

## 3.1 TLS 종료 모드

```
// gateway/src/proxy/tls_terminate.rs
impl Proxy {
```

```rust
pub(crate) async fn proxy(
&self,
inbound: TcpStream,
buffer: Vec<u8>,
app_id: &str,
port: u16,
) -> Result<()> {
// 1. 특수 앱 ID 처리
if app_id == "health" {
return self.handle_health_check(inbound, buffer, port).await;
}
if app_id == "gateway" {
return self.handle_this_node(inbound, buffer, port).await;
}


// 2. 백엔드 주소 조회
let addresses = self.get_addresses(app_id).await?;
// 3. TLS 핸드셰이크 수행
let tls_stream = self.tls_accept(inbound, buffer).await?;
// 4. 백엔드 연결
let (outbound, _counter) = timeout(
self.config.proxy.timeouts.connect,
connect_multiple_hosts(addresses, port),
```

```
)
.await
.context("timeout on connect")?
.context("failed to connect to backend")?;


// 5. 양방향 데이터 브리징
bridge(
IgnoreUnexpectedEofStream::new(tls_stream),
outbound,
&self.config.proxy,
)
.await
}


async fn tls_accept(&self, stream: TcpStream, buffer: Vec<u8>) -> Result<TlsSt
// 1. 버퍼된 데이터와 스트림 결합
let stream = if buffer.is_empty() {
stream
} else {
ChainedStream::new(Cursor::new(buffer), stream)
};


// 2. TLS Acceptor 가져오기
```

```
let acceptor = self
.acceptor
.read()
.expect("Failed to read acceptor")
.clone();
// 3. TLS 핸드셰이크 수행
let tls_stream = timeout(
self.config.proxy.timeouts.handshake,
acceptor.accept(stream),
)
.await
.context("timeout on tls handshake")?
.context("failed to accept tls connection")?;

Ok(tls_stream)
}
}
```

### 3.2 TLS 설정 생성

```
// gateway/src/proxy/tls_terminate.rs
pub(crate) fn create_acceptor(config: &ProxyConfig) -> Result<TlsAcceptor> {
// 1. 인증서와 키 파일 읽기
```

```rust
let cert_pem = fs::read(&config.cert_chain).context("failed to read certificate
let key_pem = fs::read(&config.cert_key).context("failed to read private key")?

// 2. 인증서 체인 파싱
let cert_chain = rustls_pemfile::certs(&mut cert_pem.as_slice())
.collect::<Result<Vec<_>, _>>()
.context("failed to parse certificate")?;

// 3. 개인키 파싱
let private_key = rustls_pemfile::private_key(&mut key_pem.as_slice())
.context("failed to parse private key")?
.context("no private key found")?;

// 4. TLS 설정 구성
let mut tls_config = rustls::ServerConfig::builder()
.with_no_client_auth()
.with_single_cert(cert_chain, private_key)
.context("failed to build tls config")?;

// 5. ALPN 프로토콜 설정
tls_config.alpn_protocols = vec![b"h2".to_vec(), b"http/1.1".to_vec()];
// 6. TLS 버전 설정
match config.tls_version {
```

```
TlsVersion::V1_2 => {
tls_config.versions = vec![&rustls::version::TLS12];
}
TlsVersion::V1_3 => {
tls_config.versions = vec![&rustls::version::TLS13];
}
TlsVersion::Both => {
tls_config.versions = vec![&rustls::version::TLS12, &rustls::version::TLS13];
}
}


Ok(TlsAcceptor::from(Arc::new(tls_config)))
}
```

### 3.3 TLS 패스스루 모드

```
// gateway/src/proxy/tls_passthough.rs
pub(crate) async  fn  proxy_to_app(
state: Proxy,
inbound: TcpStream,
buffer: Vec<u8>,
app_id: &str,
port: u16,
```

```
) -> Result<()> {
// 1. 백엔드 주소 조회
let  addresses = state.get_addresses(app_id).await?;
// 2. 백엔드 연결
let (outbound, _counter) = timeout(
state.config.proxy.timeouts.connect,
connect_multiple_hosts(addresses, port),
)
.await
.context("timeout on connect")?
.context("failed to connect to backend")?;

// 3. 버퍼된 데이터 전송
if !buffer.is_empty() {
outbound.write_all(&buffer).await?;
}

// 4. 양방향 데이터 브리징 (암호화된 상태로)
bridge_raw(inbound, outbound, &state.config.proxy).await
}

pub(crate) async  fn  proxy_with_sni(
state: Proxy,
```

```
inbound: TcpStream,
buffer: Vec<u8>,
sni: &str,
) -> Result<()> {
// SNI 기반 일반 프록시 (dstack 외부 서비스용)
// 구현 세부사항...
}
```

---

# 4. WireGuard VPN

## 4.1 WireGuard 설정 구조

```
// gateway/src/models.rs (추정)
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct WgConf {
pub interface: WgInterface,
pub peers: Vec<WgPeer>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct WgInterface {
pub private_key: String,
```

```
pub address: String,
pub listen_port: u16,
pub dns: Option<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct WgPeer {
pub public_key: String,
pub allowed_ips: Vec<String>,
pub endpoint: Option<String>,
pub persistent_keepalive: Option<u32>,
}
```

### 4.2 WireGuard 설정 생성

```
// gateway/src/main_service.rs (추정)
impl GatewayRpc for RpcHandler {
async fn register_cvm(self, request: RegisterCvmRequest) -> Result<RegisterCvm
// 1. CVM 인증 및 검증
let attestation = self.verify_cvm_attestation(&request).await?;
// 2. IP 주소 할당
let allocated_ip = self.allocate_ip_address().await?;
// 3. WireGuard 키 쌍 생성 (CVM용)
```

```
let (private_key, public_key) = generate_wireguard_keypair()?;

// 4. WireGuard 설정 생성

let  wg_config = WireGuardConfig {

interface: WgInterface {

private_key: private_key.clone(),

address: format!("{}/24", allocated_ip),

listen_port: 0, // 동적 할당

dns: Some("1.1.1.1".to_string()),

},

peers: vec![WgPeer {

public_key: self.state.gateway_public_key.clone(),

allowed_ips: vec!["0.0.0.0/0".to_string()],

endpoint: Some(format!("{}:{}", self.config.public_ip, self.config.wg_port)),

persistent_keepalive: Some(25),

}],

};

// 5. Gateway 측 피어 추가

self.add_wireguard_peer(&public_key, &allocated_ip).await?;

// 6. 인스턴스 정보 저장

let  instance_info = InstanceInfo {

instance_id: attestation.instance_id,

app_id: attestation.app_id,

wg_ip: allocated_ip,
```

```
wg_public_key: public_key,

last_seen: SystemTime::now(),

};

self.state.lock().instances.insert(

attestation.instance_id.clone(),

instance_info,

);

Ok(RegisterCvmResponse {

wg_config,

gateway_info: self.get_gateway_info().await?,

})

}

}
```

### 4.3 WireGuard 네트워크 관리

```
// gateway/src/main_service.rs (추정)

impl Proxy {

async  fn  add_wireguard_peer(&self, public_key: &str, ip: &Ipv4Addr) -> Result<

// 1. WireGuard 설정 파일 업데이트

let  peer_config = format!(

"[Peer]\n\

PublicKey = {}\n\
```

```
AllowedIPs = {}/32\n\n",
public_key, ip
);
// 2. wg 명령어를 통한 동적 피어 추가
cmd!(wg set wg0 peer {public_key} allowed-ips {ip}/32)?;
// 3. 설정 파일에 영구 저장
let config_path = "/etc/wireguard/wg0.conf";
let mut config = fs::read_to_string(config_path)?;
config.push_str(&peer_config);
safe_write(config_path, &config)?;
info!("Added WireGuard peer: {} -> {}", public_key, ip);
Ok(())
}
async fn remove_wireguard_peer(&self, public_key: &str) -> Result<()> {
// 1. wg 명령어를 통한 피어 제거
cmd!(wg set wg0 peer {public_key} remove)?;
// 2. 설정 파일에서 제거
// ... 설정 파일 파싱 및 업데이트 로직
info!("Removed WireGuard peer: {}", public_key);
Ok(())
}
}
```

# 5. 도메인 라우팅

## 5.1 주소 그룹 관리

```
// gateway/src/proxy.rs (추정)
#[derive(Debug, Clone)]
pub struct AddressGroup {
pub addresses: SmallVec<[AddressInfo; 4]>,
pub last_updated: Instant,
}

#[derive(Debug, Clone)]
pub struct AddressInfo {
pub ip: Ipv4Addr,
pub instance_id: String,
pub weight: u32,
}

impl Proxy {
pub async fn get_addresses(&self, app_id: &str) -> Result<AddressGroup> {
// 1. 캐시 확인
{
let state = self.lock();
```

```
if let Some((cached, timestamp)) = state.state.top_n.get(app_id) {
if timestamp.elapsed() < Duration::from_secs(30) {
return Ok(cached.clone());
}
}
}
// 2. 인스턴스 조회
let addresses = {
let state = self.lock();
let instance_ids = state.state.apps.get(app_id)
.ok_or_else(|| anyhow!("App not found: {}", app_id))?;
instance_ids
.iter()
.filter_map(|instance_id| {
state.state.instances.get(instance_id).map(|info| AddressInfo {
ip: info.wg_ip,
instance_id: instance_id.clone(),
weight: 1, // 기본 가중치
})
})
.collect::<SmallVec<_>>()
};
if addresses.is_empty() {
```

```
bail!("No healthy instances found for app: {}", app_id);
}
// 3. 주소 그룹 생성 및 캐시
let address_group = AddressGroup {
addresses,
last_updated: Instant::now(),
};
{
let mut state = self.lock();
state.state.top_n.insert(
app_id.to_string(),
(address_group.clone(), Instant::now()),
);
}
Ok(address_group)
}
}
```

## 5.2 로드 밸런싱

```
// gateway/src/proxy.rs (추정)
async fn connect_multiple_hosts(
address_group: AddressGroup,
```

```
port: u16,
) -> Result<(TcpStream, EnteredCounter)> {
let mut last_error = None;
// 1. 주소 목록 셔플 (간단한 로드 밸런싱)
let mut addresses = address_group.addresses;
use rand::seq::SliceRandom;
addresses.shuffle(&mut rand::thread_rng());
// 2. 각 주소에 순차적으로 연결 시도
for addr_info in addresses {
match TcpStream::connect((addr_info.ip, port)).await {
Ok(stream) => {
let counter = EnteredCounter::new(&NUM_BACKEND_CONNECTIONS);
info!("Connected to {}:{} (instance: {})",
addr_info.ip, port, addr_info.instance_id);
return Ok((stream, counter));
}
Err(e) => {
warn!("Failed to connect to {}:{}: {}", addr_info.ip, port, e);
last_error = Some(e);
}
}
}
Err(last_error.unwrap_or_else(|| {
```

```
anyhow!("No addresses available")
}).into())
}
```

---

# 6. 연결 관리

## 6.1 연결 카운터

```
// gateway/src/models.rs (추정)
use std::sync::atomic::{AtomicUsize, Ordering};

pub static NUM_CONNECTIONS: AtomicUsize = AtomicUsize::new(0);
pub static NUM_BACKEND_CONNECTIONS: AtomicUsize = AtomicUsize::new(0);

pub struct EnteredCounter {
counter: &'static AtomicUsize,
}

impl EnteredCounter {
pub fn new(counter: &'static AtomicUsize) -> Self {
counter.fetch_add(1, Ordering::Relaxed);
Self { counter }
```

```
}
}

impl Drop for EnteredCounter {
fn drop(&mut self) {
self.counter.fetch_sub(1, Ordering::Relaxed);
}
}

pub trait Counting {
fn current_connections(&self) -> usize;
}

impl Counting for AtomicUsize {
fn current_connections(&self) -> usize {
self.load(Ordering::Relaxed)
}
}
```

## 6.2 타임아웃 설정

```
// gateway/src/config.rs (추정)
#[derive(Debug, Clone, Deserialize)]
```

```rust
pub struct TimeoutConfig {
pub handshake: Duration, // TLS 핸드셰이크 타임아웃
pub connect: Duration, // 백엔드 연결 타임아웃
pub total: Duration, // 전체 연결 타임아웃
pub idle: Duration, // 유휴 연결 타임아웃
}

impl Default for TimeoutConfig {
fn default() -> Self {
Self {
handshake: Duration::from_secs(10),
connect: Duration::from_secs(5),
total: Duration::from_secs(300),
idle: Duration::from_secs(60),
}
}
}
```

## 6.3 데이터 브리징

```rust
// gateway/src/proxy/io_bridge.rs (추정)
pub async fn bridge<R, W>(
mut reader: R,
```

```
mut writer: W,
config: &ProxyConfig,
) -> Result<()>
where
R: AsyncRead + AsyncWrite + Unpin,
W: AsyncRead + AsyncWrite + Unpin,
{
let (mut r1, mut w1) = tokio::io::split(reader);
let (mut r2, mut w2) = tokio::io::split(writer);
// 양방향 데이터 복사
let copy1 = tokio::io::copy(&mut r1, &mut w2);
let copy2 = tokio::io::copy(&mut r2, &mut w1);
// 타임아웃과 함께 실행
let result = timeout(
config.timeouts.idle,
tokio::try_join!(copy1, copy2),
).await;
match result {
Ok(Ok((bytes1, bytes2))) => {
info!("Bridge completed: {} bytes up, {} bytes down", bytes1, bytes2);
Ok(())
}
Ok(Err(e)) => {
```

```
error!("Bridge error: {}", e);
Err(e.into())
}
Err(_) => {
warn!("Bridge timeout");
Err(anyhow!("Bridge timeout"))
}
}
}
```

---

# 7. 인증서 관리

---

## 7.1 자동 인증서 갱신

```
// gateway/src/main_service.rs (추정)
impl Proxy {
pub  async  fn  start_certificate_renewal(&self) -> Result<()> {
let  certbot = self.certbot.as_ref()
.ok_or_else(|| anyhow!("Certbot not configured"))?;
// 24시간마다 인증서 갱신 확인
let  mut  interval = tokio::time::interval(Duration::from_secs(86400));
loop {
interval.tick().await;
```

```
match certbot.renew_if_needed().await {
Ok(renewed) => {
if renewed {
info!("Certificate renewed, reloading TLS acceptor");
if let Err(e) = self.reload_certificates() {
error!("Failed to reload certificates: {}", e);
}
}
}
Err(e) => {
error!("Certificate renewal failed: {}", e);
}
}
}
}
pub fn reload_certificates(&self) -> Result<()> {
info!("Reloading TLS certificates");
let new_acceptor = create_acceptor(&self.config.proxy)?;
// TLS Acceptor 교체
let mut acceptor = self.acceptor.write()
.expect("Failed to write acceptor");
*acceptor = new_acceptor;
info!("TLS certificates reloaded successfully");
```

```
Ok(())
}
}
```

## 7.2 Let's Encrypt 통합

```
// certbot 라이브러리 사용 (추정)
impl CertBot {
pub  async  fn  renew_if_needed(&self) -> Result<bool> {
// 1. 현재 인증서 만료일 확인
let  cert_path = &self.config.cert_path;
let  cert_data = fs::read(cert_path)?;
let  cert = parse_x509_certificate(&cert_data)?;
let  expires_at = cert.validity().not_after;
let  now = SystemTime::now();
let  time_until_expiry = expires_at.duration_since(now)?;
// 2. 30일 이내 만료 시 갱신
if  time_until_expiry < Duration::from_secs(30 * 24 * 3600) {
info!("Certificate expires in {:?}, renewing", time_until_expiry);
self.renew_certificate().await?;
Ok(true)
} else {
debug!("Certificate valid for {:?}", time_until_expiry);
```

```
Ok(false)

}

}

async fn renew_certificate(&self) -> Result<()> {

// 1. ACME 클라이언트 초기화

let acme_client = AcmeClient::new(&self.config.acme_url)?;

// 2. DNS Challenge 수행

let challenge = acme_client.request_certificate(&self.config.domain).await?;

self.setup_dns_challenge(&challenge).await?;

// 3. 인증서 다운로드

let cert_chain = acme_client.finalize_order(&challenge).await?;

// 4. 인증서 파일 저장

safe_write(&self.config.cert_path, &cert_chain.certificate)?;

safe_write(&self.config.key_path, &cert_chain.private_key)?;

info!("Certificate renewed successfully");

Ok(())

}

}
```

---

# 8. 클러스터링

---

## 8.1 노드 발견

```rust
// gateway/src/main_service.rs

#[derive(Debug, Clone, Serialize, Deserialize)]

pub(crate) struct GatewayNodeInfo {

pub  id: Vec<u8>,

pub  url: String,

pub  wg_peer: WireGuardPeer,

pub  last_seen: SystemTime,

}



impl Proxy {

pub  async  fn  discover_nodes(&self) -> Result<()> {

// 1. 부트노드에서 노드 목록 조회

if  let Some(bootnode_url) = &self.config.bootnode_url {

let  client = create_gateway_client(bootnode_url)?;

let  response = client.list_nodes().await?;

// 2. 새로운 노드들 추가

let  mut  state = self.lock();

for  node  in  response.nodes {

if  node.id != self.my_node_id() {

state.state.nodes.insert(node.url.clone(), GatewayNodeInfo {

id: node.id,

url: node.url,
```

```
wg_peer: node.wg_peer,
last_seen: SystemTime::now(),
});
}
}
}
// 3. 기존 노드들과 동기화
self.sync_with_peers().await?;
Ok(())
}
async fn sync_with_peers(&self) -> Result<()> {
let nodes = {
let state = self.lock();
state.state.nodes.clone()
};
for (url, node_info) in nodes {
// 5분 이상 응답 없는 노드는 제거
if node_info.last_seen.elapsed().unwrap_or_default() > Duration::from_secs(300)
self.remove_node(&url).await?;
continue;
}
// 노드 상태 동기화
if let Err(e) = self.sync_with_node(&url).await {
```

```
warn!("Failed to sync with node {}: {}", url, e);
}
}
Ok(())
}
}
```

## 8.2 상태 동기화

```
// gateway/src/main_service.rs (추정)
impl Proxy {
async fn sync_with_node(&self, node_url: &str) -> Result<()> {
let client = create_gateway_client(node_url)?;
// 1. 노드 상태 조회
let node_state = client.get_state().await?;
// 2. 인스턴스 정보 병합
let mut state = self.lock();
for (instance_id, instance_info) in node_state.instances {
// 더 최신 정보로 업데이트
match state.state.instances.get(&instance_id) {
Some(existing) if existing.last_seen > instance_info.last_seen => {
// 기존 정보가 더 최신
continue;
```

```
}
_ => {
// 새 정보로 업데이트
state.state.instances.insert(instance_id.clone(), instance_info.clone());
// 앱 매핑도 업데이트
state.state.apps
.entry(instance_info.app_id.clone())
.or_default()
.insert(instance_id);
}
}
}
// 3. 노드 최종 접촉 시간 업데이트
if  let Some(node_info) = state.state.nodes.get_mut(node_url) {
node_info.last_seen = SystemTime::now();
}
Ok(())
}
}
```

---

## 요약

dstack Gateway의 네트워킹 시스템은 다음과 같은 특징을 가집니다:

### 핵심 기능:

1. **TLS 종료/패스스루**: 유연한 TLS 처리 모드
2. **도메인 기반 라우팅**: SNI를 통한 자동 라우팅
3. **WireGuard VPN**: CVM과의 안전한 네트워크 연결
4. **로드 밸런싱**: 다중 인스턴스 간 트래픽 분산
5. **자동 인증서 관리**: Let's Encrypt 통합
6. **클러스터링**: 다중 Gateway 노드 지원

### 네트워크 플로우:

1. **클라이언트 연결** → **SNI 추출** → **도메인 파싱**
2. **TLS 처리** → **백엔드 조회** → **연결 설정**
3. **데이터 브리징** → **연결 모니터링** → **정리**

### 보안 특성:

- **End-to-End 암호화**: TLS 패스스루 모드 지원
- **네트워크 격리**: WireGuard를 통한 CVM 격리
- **인증서 검증**: 자동 갱신 및 검증
- **접근 제어**: 인증된 CVM만 네트워크 접근 허용

이 시스템을 통해 dstack은 인터넷과 TEE 환경 사이의 안전하고 효율적인 네트워크 브리지 역할을 수행합니다.